Docker 网络实现 · 实战

第 8 讲

目标

上一讲的理论, 动手做成三件事

- 外部访问: `-p / -P` 端口映射各种形态
- 容器互联: `--link` 对比**自定义网络**
- 配置 DNS: `--dns / --dns-search`
- 完成"两容器互联 + 外部访问"实验并截图

一、外部访问容器

> 出口要开个"门"

隔着网段, 外部如何进得来?

> 容器IP是内网网段(172.17.x.x), 外部摸不到 答案是启动时用 `-p` 把宿主端口绑到容器端口

-p 前是宿主机, 后是容器

```
docker run -d -p 8080:80 nginx
#       宿主机8080 → 容器80
curl http://localhost:8080
```



> 流量先到宿主机8080, 再被 iptables 转进容器80

不写端口？就随机送你一个

```
docker run -d -P nginx              # 随机映射所有EXPOSE端口
docker run -d -p 80 nginx          # 只映射80, 宿主端口随机
docker ps    # PORTS: 0.0.0.0:32768→80/tcp
```



> 随机端口适合临时看一眼, 固定端口才对外

还能绑定地址、映射多端口

```
docker run -d -p 127.0.0.1:8080:80 nginx   # 只许本机
docker run -d -p 80:80 -p 443:443 myapp    # 多端口混用
```



> 左地址决定"绑定哪张网卡", 多写几个 -p 就开几扇门

二、容器互联

> 让容器按"名字"找到彼此

互联不是记IP, 是认一个"稳定名字"

- 容器IP会变, 不适合写死
- 期待: 用一个名字, 稳定地找到另一台容器

> 两条路: 古老的 `--link` , 现代的自定义网络

方式一: --link (旧式)

```
docker run -d --name db mysql
docker run -d --name web --link db:db nginx
docker exec -it web ping db     # web 认识 db
```



- 在 web 的 hosts 里**手动写一条** db→IP 的映射
- **单向**: 只对挂了 --link 的容器生效

方式二: 自定义网络 (推荐)

```
docker network create appnet
docker run -d --name db  --network appnet mysql
docker run -d --name web --network appnet nginx
docker exec -it web ping db            # 直接 ping 名字
docker exec -it db getent hosts web    # 反过来也认(双向)
```



相关容器接进同一张网, 名字自动当域名

一表看清两条路

维度	-link	自定义网络
方向	单向、点对点	**双向、任意到任意**
解析	写死在 hosts	内嵌 **DNS**
新增容器	要重建/加link	自动发现
归属	旧式, 了解	**官方推荐**

三、配置 DNS

> 容器问谁要地址?

容器默认继承宿主机的 DNS 设置

不加配置, resolv.conf 的 nameserver 就是宿主机的 想固定/想换目标 → 给容器显式指定

-dns 指定服务器, --dns-search 设搜索域

```
docker run -d --name web \
  --dns 8.8.8.8 \
  --dns-search mycorp.example \
  nginx

docker exec -it web cat /etc/resolv.conf
# nameserver 8.8.8.8
# search mycorp.example
```



四、实战一

外部访问容器

> 把 -p 的花样都过一遍

固定、随机、本机、多端口

```
docker run -d --name web -p 8080:80 nginx
curl localhost:8080                # 固定端口 ✓

docker run -d --name web2 -P nginx
docker ps                          # 看 PORTS 随机端口
curl localhost:那个随机端口        # 能访问 ✓

docker run -d --name web3 -p 127.0.0.1:8081:80 nginx
docker run -d --name multi -p 80:80 -p 8000:80 nginx
```



五、实战二

容器互联 (作业)

> -link 与自定义网络, 再对比

web 找 db: 旧式与推荐各来一遍

```
# A: --link
docker run -d --name db --network legacynet mysql:8
docker run -d --name web --network legacynet --link db:db nginx
docker exec -it web cat /etc/hosts    # 有 db 条目
docker exec -it web ping db           # 通 ✓

# B: 自定义网络
docker run -d --name db --network appnet mysql:8
docker run -d --name web --network appnet nginx
docker exec -it web ping db           # 也通 ✓
docker exec -it db getent hosts web   # 双向 ✓
```



六、实战三

配置 DNS

显式指定与默认对比

配了显式的, 就听显式的

```
docker run -d --name dnsbox \
  --dns 8.8.8.8 --dns-search example.test nginx
docker exec -it dnsbox cat /etc/resolv.conf
# nameserver 8.8.8.8, search example.test

docker run -d --name defaultbox nginx
docker exec -it defaultbox cat /etc/resolv.conf
# 继承宿主机的 nameserver
```



一张表带走全部要点

方面	要点
外部访问	启动时 `-p` 映射; `-P` 随机; 可绑地址、可多端口; **启动后改不了**
端口读法	`-p 前宿主机、后容器`; 写死固定, 不写随机
容器互联	`--link` (单向) vs **自定义网络**(双向、DNS)
DNS	`--dns` nameserver · `--dns-search` 搜索域 · 不配则继承宿主
命令	`-p/-P` · `network create/connect` · `ps` 看 PORTS

容器能连起来了, 可分写进去会"一键清零"

> 下一步: 数据卷与持久化 —— 让容器里的数据不随删除而消失

谢谢大家

Q & A